T3000 API & Protocol

Message 14: SAVE_NEW_LIBRARY_DATA · built Nov 13, 2026

Message 14: SAVE_NEW_LIBRARY_DATA

The SAVE_NEW_LIBRARY_DATA message saves the new T3 HVAC equipment library data to a JSON file on the server.

When to Use:

- Saving T3 HVAC library
- Persisting updated equipment configurations
- Backing up library changes

Overview

Action: `SAVE_NEW_LIBRARY_DATA` (14) **Direction:** Frontend → Backend **Location:** BacnetWebView.cpp line 2990 **Purpose:** Save T3 HVAC library JSON file

Request Format

JSON Structure:

JSON

```
{
  "action": "SAVE_NEW_LIBRARY_DATA",
  "data": {
    "version": "2.0",
    "equipment": [
      {
        "id": "ahu-001",
        "name": "Air Handler Unit 1",
        "category": "HVAC",
        "points": [...]
      }
    ]
  }
}
```

Request Parameters

Parameter	Type	Required	Description
`data`	object	Yes	Complete T3 library data structure

Response Format

JSON Structure:

```JSON
{
  "action": "SAVE_NEW_LIBRARY_DATA_RES",
  "status": true
}
```

Implementation

Location: BacnetWebView.cpp line 2990

```C++
case SAVE_NEW_LIBRARY_DATA:
{
    CString temp_lib_file = _T("t3_hvac_library.json");
    des_lib_file = image_fordor + _T("\\") + temp_lib_file;
    const std::string file_output = Json::writeString(builder, json["data"]);

    CFile file;
    CString file_temp_cs(file_output.c_str());
    file.Open(des_lib_file, CFile::modeCreate | CFile::modeWrite, NULL);
    file.Write(file_temp_cs, file_temp_cs.GetLength() * 2);
    file.Close();

    Json::Value tempjson;
    tempjson["action"] = "SAVE_NEW_LIBRARY_DATA_RES";
    tempjson["status"] = true;

    const std::string output = Json::writeString(builder, tempjson);
    outmsg = CString(output.c_str());
}
```

File Location: `t3_hvac_library.json`

Difference from SAVE_LIBRARY_DATA

Message	File	Purpose
SAVE_LIBRARY_DATA (10)	`hvac_library.json`	Legacy library format
SAVE_NEW_LIBRARY_DATA (14)	`t3_hvac_library.json`	New T3 library format

Frontend Usage

TYPESCRIPT

```typescript
const saveT3Library = async (libraryData: any): Promise<boolean> => {
  const message = {
    action: 'SAVE_NEW_LIBRARY_DATA',
    data: libraryData
  };

  sendMessage(JSON.stringify(message));

  return new Promise((resolve, reject) => {
    const handler = (data: string) => {
      const response = JSON.parse(data);

      if (response.action === 'SAVE_NEW_LIBRARY_DATA_RES') {
        resolve(response.status);
        webSocket.off('message', handler);
      }
    };

    webSocket.on('message', handler);

    setTimeout(() => {
      webSocket.off('message', handler);
      reject(new Error('Save timeout'));
    }, 5000);
  });
};

// Usage
try {
  const success = await saveT3Library(libraryData);
  if (success) {
    showNotification('Library saved successfully');
  }
} catch (error) {
  showError('Failed to save library');
}
```

Auto-save

```typescript
const LibraryEditor: React.FC = () => {
  const [library, setLibrary] = useState<any>({});
  const [saving, setSaving] = useState(false);

  // Auto-save after 2 seconds of inactivity
  useEffect(() => {
    const timer = setTimeout(async () => {
      setSaving(true);
      try {
        await saveT3Library(library);
      } finally {
        setSaving(false);
      }
    }, 2000);

    return () => clearTimeout(timer);
  }, [library]);

  return (
    <div>
      {saving && <span>Saving...</span>}
      {/* Library editor UI */}
    </div>
  );
};
```

See Also

- SAVE_LIBRARY_DATA - Save legacy library